GovSkill.

Sovereign Civic Platform & Digital Document Intelligence

Complete Technical Dossier, Fresh Machine Setup & College Viva / Interview Master Guide

GITHUB REPOSITORY https://github.com/sachin-saroj/GovSkill	LIVE FRONTEND WEB APPLICATION https://govskill-frontend.onrender.com
LIVE BACKEND API PROBE https://govskill-backend.onrender.com/health	INTERACTIVE SWAGGER DOCUMENTATION https://govskill-backend.onrender.com/docs
CLOUD INFRASTRUCTURE DASHBOARD dashboard.render.com (GovSkill Project)	AUTOMATED QUALITY GATES 119 Backend Tests (Pytest) • 121 Frontend Tests (Vitest) • 100% Passing

Executive Summary: GovSkill is a full-stack digital skill support and statutory document pre-validation platform built for local municipal and revenue administration offices. It bridges two critical gaps: (1) *Employee Competency* through structured curriculum, server-scored examinations, dual-mode grounded AI mentoring, and HMAC-SHA256 tamper-evident credentials; and (2) *GovAssist Citizen Self-Service* via client-side OCR extraction, a 100% deterministic 4-rule validation engine, and printable anti-fraud counter slips with cryptographic QR tracking.

Section 1: Friend's Laptop Setup Guide (A to Z)

Follow these exact step-by-step instructions to run GovSkill from scratch on any Windows, macOS, or Linux computer.

Step 0: Prerequisites Installation

Make sure the friend's laptop has the following three essential development tools installed:

Software	Recommended Version	Verification Command
Git	2.35+	`git --version`
Python	3.11 or 3.12 (or 3.13)	`python --version`
Node.js & npm	Node 18+ or 20+ (LTS)	`node -v` and `npm -v`

Step 1: Clone the GitHub Repository

Open Terminal or PowerShell in the preferred folder (e.g. Desktop) and run:

```
git clone https://github.com/sachin-saroj/GovSkill.git
cd GovSkill
```

Step 2: Backend Environment Setup

Configure the Python FastAPI virtual environment and install dependencies:

```
# 1. Navigate to backend directory
cd backend

# 2. Create isolated Python virtual environment
python -m venv venv

# 3. Activate the virtual environment
# On Windows (PowerShell):
.□env\Scripts□ctivate
# On macOS / Linux:
source venv/bin/activate

# 4. Install exact pinned dependencies
pip install -r requirements.txt

# 5. Create backend .env configuration from example
copy .env.example .env     # (On macOS/Linux use: cp .env.example .env)

# 6. Apply database migrations to generate SQLite tables
alembic upgrade head
```

> **Automatic Database Creation:** Running `alembic upgrade head` automatically initializes the local SQLite database file (`govskill.db`) with all 8 versioned migrations up to `008_add_user_age` . No PostgreSQL installation or database passwords are required!

Step 3: Frontend Environment Setup

In a new terminal window or tab, install the React frontend packages:

```
# Navigate to the frontend directory
cd frontend

# Install node dependencies
npm install
```

Step 4: Launching the Application (Two Methods)

Method A: 1-Click Dev Launcher (Recommended on Windows)

Double-click the file `start_dev.bat` located in the root of the `GovSkill` folder. It automatically:

- Activates Python venv and starts the FastAPI Uvicorn backend on `http://localhost:8000` .
- Starts the Vite React frontend development server on `http://localhost:3000` .

Method B: Manual Terminal Execution (Any OS)

- **Terminal 1 (Backend API):**

  ```
  cd backend
  .□env\Scripts□ctivate
  uvicorn app.main:app --reload --port 8000
  ```

- **Terminal 2 (Frontend Client):**

  ```
  cd frontend
  npm run dev
  ```

Step 5: Testing & Verification

- **Open Web Application:** http://localhost:3000
- **Login / Registration Portal:** http://localhost:3000/login
- **Interactive Swagger API Docs:** http://localhost:8000/docs
- **API Health Check Probe:** http://localhost:8000/health

Troubleshooting Common Setup Issues

- **PowerShell Script Execution Error:** If Windows blocks activating the venv, run: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`
- **Port 8000 or 3000 Already in Use:** Check running processes with: `Get-Process -Id (Get-NetTCPConnection -LocalPort 8000).OwningProcess | Stop-Process`

Section 2: System Architecture & Technical Plumbing (A to Z)

1. High-Level Technology Stack

Layer	Technology	Purpose & Architectural Role
Frontend Client	React 18 + Vite + TypeScript	Single Page Application (SPA), strict typing, route-level code splitting.
UI Styling	Tailwind CSS 3	Museum-grade civic editorial tokens (warm ivory paper `#FAF8F2`, Fraunces serif).
HTTP Client	Axios	Custom instance with 45s cloud timeout, JWT request interceptor, friendly error handling.
Backend REST API	FastAPI (Python 3.11+)	High-concurrency async ASGI engine, OpenAPI 3.0 auto-documentation.
Validation	Pydantic v2	Strict request/response payload schemas and environment configuration parsing.
Database Engine	SQLAlchemy 2.0 Async	Fully async ORM with connection pooling, multi-DB dialect support.
Database Dialects	PostgreSQL 16 & SQLite	Dual-strategy: PostgreSQL on cloud with automatic graceful SQLite fallback.
Database Migrations	Alembic	Strict version-controlled schema evolution (001 to 008 revisions).
Authentication	JWT (python-jose + bcrypt)	Stateless authentication tokens, bcrypt password hashing (cost ≥ 12), admin 2FA OTP.
Document OCR	Tesseract OCR + Pillow + PyMuPDF	Multi-format (PDF/PNG/JPG) rendering, contrast-boosting binarization, regex parsing.
AI Integration	Google Gemini API (gemini-2.5-flash)	Isolated behind centralized AIGateway for grounded tutoring & failure explanation.

2. Database Layer Architecture (How, Where & What)

GovSkill employs an enterprise **Dual-Engine Resilient Database Architecture**:

> **Where is the database located?**
>
> - **Local Environment:** Stored in a lightweight, zero-dependency SQLite file at `backend/govskill.db` using the `sqlite+aiosqlite:///./govskill.db` driver.
> - **Production Cloud (Render):** Primary target is a managed PostgreSQL 16 cluster. If the cloud database is paused, suspended, or unreachable, the container's `entrypoint.sh` automatically activates an internal SQLite database fallback, guaranteeing zero downtime.

Core Database Relational Entities:

Table Name	Primary Key	Key Attributes & Purpose
`users`	UUID	`email`, `password_hash`, `role` (employee/admin), `age`, `token_version`, `is_active`.
`modules`	UUID	`title`, `content` (Markdown administrative training curriculum).
`user_progress`	UUID	Foreign keys to user & module, `current_section_index`, `lessons_completed`, `best_score`, `status`.
`quiz_questions`	UUID	`module_id`, `question`, `options` (JSON), `correct_option_index` (server-only!), `competency`.
`quiz_attempts`	UUID	`user_id`, `module_id`, `score`, `total_questions`, `passed` (threshold ≥ 75%).
`credentials`	UUID	Unique human-readable `credential_id` (`GS-CERT-YYYY-HEX`), `verification_hash`, HMAC-SHA256 `signature`.
`citizen_documents`	UUID	`filename`, `file_hash`, `document_type`, `extracted_data` (JSON), `rules_breakdown` (JSON). *Isolated from users table!*
`email_otps`	UUID	`otp_hash`, `purpose` ("login_2fa"), 5-minute timed expiration for admin security.

3. End-to-End Workflow Connections

Workflow A: GovAssist Citizen Document Pre-Validation

1. Citizen uploads document file (PDF, PNG, JPG) at `/citizen` (unauthenticated).
2. Vite sends multipart payload to `POST /api/documents/upload`. Rate limiter validates client IP.
3. Backend checks file extension and 5 MB size limit. PyMuPDF renders PDF pages to images.
4. Pillow preprocessing applies grayscale conversion, 2.0x contrast boosting, and threshold binarization.
5. Tesseract OCR extracts raw text. Keyword classifier detects document type (15 civic types).
6. **Deterministic 4-Rule Engine** executes in Python:
   - Rule 1: Applicant Name Presence & Validity
   - Rule 2: Certificate Number Pattern Regex

    - Rule 3: Certificate Validity & Date Expiration Check
    - Rule 4: Issuing Authority & Official Seal Completeness
7. If rules fail, Gemini AI generates plain-language, non-hallucinating remedial instructions.
8. Frontend renders interactive Counter Slip Modal with dynamic cryptographic verification QR code for physical presentation at the municipal counter.

Workflow B: Civil Servant Qualification & Cryptographic Certification

1. Employee registers and authenticates via `POST /api/auth/login`, receiving an encrypted JWT.
2. Employee accesses administrative curriculum at `/module`, tracking reading progress.
3. When stuck, the employee queries the **AI Tutor**. In `grounded_training` mode, the system searches curriculum sections and strictly refuses out-of-scope inquiries to prevent hallucination.
4. Employee attempts the 8-question competency examination at `/quiz`. Answer keys (`correct_option_index`) are **never exposed** in API responses.
5. Submission sent to `POST /api/quiz/{id}/submit` for authoritative server-side scoring.
6. Upon achieving ≥ 75%, an official `Credential` record is atomically minted with an HMAC-SHA256 cryptographic digest.
7. Anyone can publicly verify the certificate at `/verify/:credentialId` without authenticating.

Section 3: College Viva & Technical Interview Master Guide

These 16 questions cover the toughest architectural, security, and design decisions in GovSkill. Memorize the core technical rationales to sound authoritative and confident before examiners.

Q1: Explain the product vision and core architecture of GovSkill in 60 seconds.

"GovSkill is a sovereign full-stack civic platform designed for municipal and revenue offices. It solves two problems in one unified system: first, **Civil Servant Qualification** through interactive curriculum reading, grounded AI tutoring, server-evaluated competency exams, and HMAC-SHA256 digitally signed credentials; second, **GovAssist Citizen Pre-Submission Validation**, which allows citizens to upload documents before visiting municipal counters to run OCR extraction, a deterministic 4-rule validation engine, and generate printable counter slips with verification QR codes. The tech stack is FastAPI on Python 3.11, React 18 with Vite and TypeScript, PostgreSQL with automatic SQLite fallback, and Google Gemini 2.5 Flash for grounded AI assistance."

Q2: Why did you build the Document Validation Engine using deterministic Python code rather than letting an LLM (Gemini/GPT) decide pass or fail?

"This is our most important architectural decision. **LLMs are probabilistic and prone to hallucinations**; they can never be trusted to make statutory legal decisions. A fake certificate could slip through or a valid certificate could be rejected based on prompt phrasing. In GovSkill, validation pass/fail decisions are **100% deterministic Python code**: regex pattern checks, datetime expiration comparisons, and seal presence algorithms. The AI layer (Gemini) is strictly isolated to an *explainer role*: it only generates plain-language remediation instructions for rules that our Python code has *already* determined to have failed."

Q3: Why did you choose FastAPI over Django, Flask, or Node.js / Express?

"Three reasons: (1) **High Asynchronous Concurrency:** FastAPI is built on Starlette and ASGI, utilizing Python's `async/await` event loop. Document OCR and AI calls are I/O bound; async handling prevents blocking worker threads under concurrent citizen uploads. (2) **Automatic Schema Validation:** With Pydantic v2, every request and response is strictly validated at runtime with zero boilerplate. (3) **Self-Documenting API:** FastAPI automatically generates interactive OpenAPI 3.0 Swagger UI at `/docs`, accelerating frontend integration and debugging."

Q4: How does your examination system prevent candidates from cheating via Browser DevTools?

"In many naive student projects, the full quiz JSON with correct answers is sent to the browser and graded in JavaScript. In GovSkill, **the answer key (`correct_option_index`) is strictly stripped server-side** before transmitting questions to the client. The candidate's browser only receives the question text and option strings. Grading takes place exclusively on the backend in `/api/quiz/{id}/submit` against the authoritative database records."

Q5: How does your cryptographic digital credential signing work? Can a user forge a certificate?

"No, certificates cannot be forged. When an employee scores ≥ 75%, the backend canonicalizes the credential metadata (user ID, module ID, score, issuance timestamp) into a normalized JSON payload. It hashes this payload using SHA-256 and generates a cryptographic signature using **HMAC-SHA256** backed by a dedicated server-only `CREDENTIAL_SIGNING_KEY`. When someone checks `/verify/:credentialId`, the server recomputes the signature. If a single byte of the score, name, or date was tampered with, the cryptographic signature fails validation immediately."

Q6: How does the Dual-Mode AI Mentor work, and how do you prevent curriculum hallucination?

"We implemented a dual-mode conversational architecture behind a centralized `AIGateway`: (1) `GROUNDED_TRAINING` Mode: Strictly bounded to municipal curriculum folios. We use code-driven keyword overlap scoring (`find_relevant_modules`) to match circulars and inject them into the system prompt. If a user asks something out-of-scope (e.g., 'give me a cake recipe'), the model strictly refuses and redirects to departmental topics. (2) `GENERAL_CHAT` Mode: A multilingual conversational assistant supporting English, Hindi (Devanagari), and Hinglish for general governance concepts and code drafting, maintaining multi-turn context memory."

Q7: Explain your dual-database architecture (PostgreSQL + SQLite fallback). What problem does it solve?

"Production cloud systems often fail due to database connectivity issues (e.g., Render free-tier databases suspend after 30 days). In GovSkill, we use SQLAlchemy 2.0 Async which abstracts database dialects cleanly. In our Docker `entrypoint.sh`, the container first probes PostgreSQL migrations. If PostgreSQL is suspended or unreachable, it dynamically catches the failure and activates an internal SQLite fallback (`sqlite+aiosqlite:///./govskill.db`), runs migrations, and boots Uvicorn. This ensures 100% uptime for presentations and evaluation without manual intervention."

Q8: What image preprocessing techniques did you use to handle blurry or low-contrast mobile certificate photos?

"Raw mobile photos of revenue certificates often suffer from uneven shadows, low lighting, and yellowish paper. We implemented a **Pillow image preprocessing pipeline**: (1) Convert the image to 8-bit grayscale to discard color noise. (2) Apply `ImageEnhance.Contrast` with a 2.0x factor to sharpen letterforms against the background. (3) Apply adaptive threshold binarization to produce high-contrast black-and-white pixels before passing the buffer to Tesseract OCR. (4) If local OCR extraction yields low confidence, our pipeline falls back to Gemini Vision OCR."

Q9: How does GovSkill comply with India's Digital Personal Data Protection (DPDP) Act 2023?

"We designed privacy directly into our data architecture: (1) **Zero Citizen Registration:** Citizens pre-check documents without creating accounts, eliminating unnecessary demographic profiling. (2) **PII Isolation:** The `citizen_documents` table has **no foreign key** to the `users` table. (3) **Sensitive Identifier Masking:** Aadhaar numbers are automatically masked as `XXXX-XXXX-1234` and PAN numbers as `ABCDE****F` across all logs, API responses, and database records."

Q10: How do you prevent Denial-of-Service (DoS) and brute-force attacks on your API?

"We implemented sliding-window in-memory rate limiters (`InMemoryRateLimiter`). Public document uploads are capped at 20 requests/minute, login attempts at 15/minute per IP and 5/minute per email, and credential verification at 30/minute. Crucially, our rate limiter resolves client IP via `get_client_ip` which only trusts `X-`

`Forwarded-For` headers when originating from verified loopback reverse proxies (Docker/Render), preventing header spoofing."

Q11: What is Alembic, and why did you use it instead of recreating database tables on startup?

"Using `Base.metadata.create_all()` in production is dangerous because it cannot handle schema alterations, column additions, or index changes on existing tables without data loss. Alembic provides **version-controlled, incremental migrations**. Every schema change (such as adding the `age` column in migration `008_add_user_age`) is recorded as a reversible migration script that can be audited, executed, and rolled back across environments."

Q12: Why React 18 with Vite instead of Next.js or a traditional Multi-Page Application (MPA)?

"GovSkill is an internal dashboard and citizen kiosk portal where client-side interactivity, immediate UI feedback, and offline state stability are paramount. Next.js introduces unnecessary server-side rendering (SSR) complexity and Node server dependencies. Vite provides instant hot-module-replacement (HMR), lightning-fast builds via Rollup/esbuild, and clean SPA routing. Furthermore, we implemented `React.lazy()` code splitting to decouple standalone pages (`AdminDashboardPage` and `CitizenUploadPage`) into on-demand asynchronous chunks, reducing the initial JS bundle by 13%."

Q13: How did you handle cloud cold starts on Render free tier?

"Free-tier container hosts spin down idle services after 15 minutes, requiring 30–45 seconds to reboot. If the client timeout is standard (10–15 seconds), requests abort before the server finishes waking up. In `api.ts`, we extended the Axios timeout to **45 seconds**, and in `apiError.ts`, we built an interceptor that detects `ECONNABORTED` or timeout codes and presents friendly guidance ('Server is waking up from cold sleep — please wait a moment') rather than raw technical errors."

Q14: What testing methodology and quality gates did you implement in this project?

"We enforced strict quality gates before every release: (1) **Backend:** 119 Pytest integration and unit tests covering JWT auth, document OCR fixtures (valid, expired, missing-field PDFs/PNGs), rate limiting, quiz scoring, and AI Gateway routing. (2) **Frontend:** 121 Vitest unit tests across 23 test suites covering interactive UI components, login forms, error banners, and automated accessibility checks via `jest-axe`. (3) **Linting & CI/CD:** GitHub Actions workflow automatically executes `ruff check .`, pytest, vitest, and `vite build` on every push."

Q15: If 10,000 citizens upload certificates simultaneously, how would you scale this architecture?

"Currently, GovSkill is designed as a modular monolith. Under massive scale: (1) **Asynchronous Task Queue:** Decouple OCR processing into background worker queues using Celery or ARQ backed by Redis, returning a `job_id` to the citizen immediately. (2) **Horizontal Worker Scaling:** Scale stateless FastAPI/Uvicorn containers horizontally behind an AWS ALB or Nginx reverse proxy. (3) **Read-Replica Database:** Separate public credential lookups (`GET /verify`) to read-replicas while keeping write operations on PostgreSQL primary. (4) **Object Storage:** Offload document uploads from local disk to S3/Google Cloud Storage with signed URLs."

Q16: What is the significance of the Museum-Grade Editorial Design System in a government application?

"Traditional government software is notoriously clunky, intimidating, and poorly designed. We built GovSkill with an **archival museum-grade editorial aesthetic**: a warm ivory canvas (`#FAF8F2`), architectural hairline rules (`#D9CFBB`), Fraunces display serif typography, and JetBrains Mono telemetry. This creates immediate civic authority, legibility for extended reading of circulars, and trust for citizens reviewing official documents."

Section 4: Production Deployment & Verification Summary

Live Deployment Architecture on Render

GovSkill is deployed as an Infrastructure-as-Code (IaC) specification via `render.yaml`:

Service Component	Runtime & Specs	Production URL	Active Status
GovSkill Frontend	Static Site (Vite Single Page App)	govskill-frontend.onrender.com	`200 OK • Deployed`
GovSkill Backend API	Docker Container (Python 3.11 + Tesseract)	govskill-backend.onrender.com	`200 OK • Healthy`
Database Engine	SQLite (Fallback Resilient) / PostgreSQL	Internal async connection	`Connected • Synced`

Git Commits & Audit Trail

- **Latest Commit:** `d61948d` — *fix(deploy): add resilient SQLite fallback on container boot, increase client timeout to 45s, and improve timeout error handling*
- **Repository Synchronization:** 100% synchronized with `origin/main` at `https://github.com/sachin-saroj/GovSkill`.
- **Automated CI Status:** GitHub Actions run `41318750476` completed with zero failures across all jobs.

> **Examiner / Interview Takeaway:** GovSkill demonstrates real-world software engineering discipline: deterministic business rules separated from probabilistic AI, cryptographic certificate security, privacy compliance by design, comprehensive automated testing, and resilient fault-tolerant cloud architecture.